

k) DateTime.Parse() vs. DateTime.ParseExact()

Both methods convert a string representation of a date into a DateTime object, but they differ in how strict they are:

- **DateTime.Parse():** This is the "smart" or "flexible" parser. It attempts to interpret the string based on the current culture settings of the system (e.g., en-US vs en-GB).
 - **Appropriate when:** You are receiving input in a standard format and you want the system to handle common variations automatically.
- **DateTime.ParseExact():** This is the "strict" parser. It requires the input string to match a specific format string (like "yyyy-MM-dd") exactly. If the string deviates by even a single character, it throws a FormatException.
 - **Appropriate when:** You have a fixed data format (like an ISO 8601 string from an API or a legacy database) and you want to ensure the data is strictly validated.

l) The "Subtract One" Age Calculation Logic

When calculating age, you typically subtract the birth year from the current year (

$Age = CurrentYear - BirthYear$). However, this is often incorrect by one year.

The Scenario:

Imagine today is **March 24, 2026**, and a patient was born on **December 31, 2000**.

1. A simple subtraction ($2026 - 2000$) suggests the patient is **26 years old**.
2. However, because their birthday hasn't occurred yet in the current year, they are actually still **25**.

The Logic:

You must check if the patient's birthday has already passed in the current year. If $CurrentDate < BirthdayThisYear$, you must **subtract one** from the year difference to get the correct legal age.

m) Case Sensitivity in Date Format Strings

C# format strings are case-sensitive because different letters represent different time units:

- **"DD/MM/YYYY":** This will likely fail or produce unexpected results. In C#, dd is for days, but DD is not a standard specifier. YYYY is generally accepted as yyyy, but standard practice uses lowercase.
- **"dd/mm/yyyy":** This produces a **wrong answer**.
 - dd = Day
 - mm = **Minutes** (00-59)
 - yyyy = Year
 - If you use mm, your date string will display the current minute of the hour instead of the month. Always use **MM** for Month.

n) Calculating Days Until the Next Birthday

To find the days remaining until a birthday, you need to determine when the *next* birthday occurs and subtract the current date from it.

Logic Steps:

1. Create a `DateTime` object for the birthday in the current year.
2. Compare it to today's date.
3. If the birthday has already passed this year, add one year to that date (to get next year's birthday).
4. Subtract `DateTime.Today` from that target date to get a `TimeSpan`.
5. Access the `.Days` property of that `TimeSpan`.

Example Code Snippet:

```
DateTime today = DateTime.Today;
DateTime nextBirthday = new DateTime(today.Year, birthDate.Month, birthDate.Day);

if (nextBirthday < today) {
    nextBirthday = nextBirthday.AddYears(1);
}

int daysUntilBirthday = (nextBirthday - today).Days;
Console.WriteLine($"Days until next birthday: {daysUntilBirthday}");
```